QTC

ARCHITECTURE & STRATEGY

QTC AI Architecture

Building a Powerful and Secure AI-Native Blockchain

AI recommends, detects and optimizes.
Deterministic consensus rules decide chain validity.

01

SECURE

Sentinel + Eclipse Guardian

02

COMPUTE

AI Jobs + verifiable inference

03

SETTLE

QTC + institutional applications

Executive and technical design brief

24 September 2026 | Version 1.0 | Proposed architecture

A development blueprint based on the supplied QTC discussion.
Implementation, performance and deployment status remain to be verified.

01 / EXECUTIVE SUMMARY

Build capability around a stable core

Make QTC useful for AI while preserving independent, reproducible validation.

QTC can develop into a network for verifiable computation and settlement by combining three capabilities: a deterministic blockchain core, an isolated AI security layer, and a commercial compute marketplace. The opportunity is to improve network resilience and create demand for compute services without giving a model authority over money or chain validity.

> **THE GOVERNING PRINCIPLE**
>
> AI may recommend, detect and optimize. Deterministic consensus rules alone decide block validity, transaction validity and canonical chain selection. Human-governed software releases may change those rules through an explicit activation process; a live model may not.

Launch two priorities together

Priority	Initial product	Evidence of success
P0 / QTC Sentinel	Observe peers, propagation, forks and node health. Start in shadow mode; introduce bounded local actions only after validation.	Useful, explainable alerts; measured false positives; no change to consensus results.
P0 / QTC AI Jobs	A narrow marketplace for reproducible matrix tasks and selected inference jobs, with explicit verification and payment terms.	Jobs complete, verify, pay or refund correctly; worker failures do not impair the node.

What this enables

Sentinel creates an immediate security workstream. AI Jobs tests real customer demand, provider economics and verification costs. Hardware portability, transaction simulation and signed oracle reports then support a broader application ecosystem. ZKML and useful AI-PoW integration remain gated by proof economics and security evidence.

Scope and naming

This brief uses **QTC** throughout, as requested, although the source conversation also used QTS. It describes a target design rather than an audit of existing software. No repository, live network or QTC benchmark was inspected. Smart contracts, proof verification, escrow and asset features require compatibility review against the actual chain.

02 / DESIGN PRINCIPLES

Define authority before capability

Keep the trusted computing base small, explicit and independently testable.

01 / Deterministic validity. Nodes must agree from the same serialized inputs and activated protocol version. No LLM response, model score, external API call or local observation belongs in the validity predicate.

02 / Separate execution domains. Consensus, AI security, commercial workers and signing run with distinct permissions and resource limits. A model outage must leave baseline node operation available.

03 / Verify the stated computation. Bind every proof to an exact program or model, input commitment, output and job identity. State clearly when assurance depends on a verifier, a committee or an operator.

04 / Preserve ordinary-node participation. Maintain an independent CPU reference verifier. Publish maximum validation cost and memory limits; accelerator access must not become a prerequisite for full validation.

05 / Bound operational automation. Use approved policies, action budgets, minimum connectivity and rollback paths. Treat peer risk scores as evidence for investigation, never as sufficient proof of misconduct.

06 / Make decisions accountable. Version models, policies, proof circuits and signed releases. Record evidence, the deciding rule, the action and the responsible operator without publishing private job data.

Three decisions, three owners

Decision	Decision authority	AI contribution
Is this chain valid?	Activated deterministic protocol rules.	None in the validity decision.
How should this node respond?	Local policy within approved limits; operator escalation.	Detection, ranking and recommendations.
Should an application trade or pay?	Published application rules and authorized signers.	Risk analysis and explanations.

Reading guide

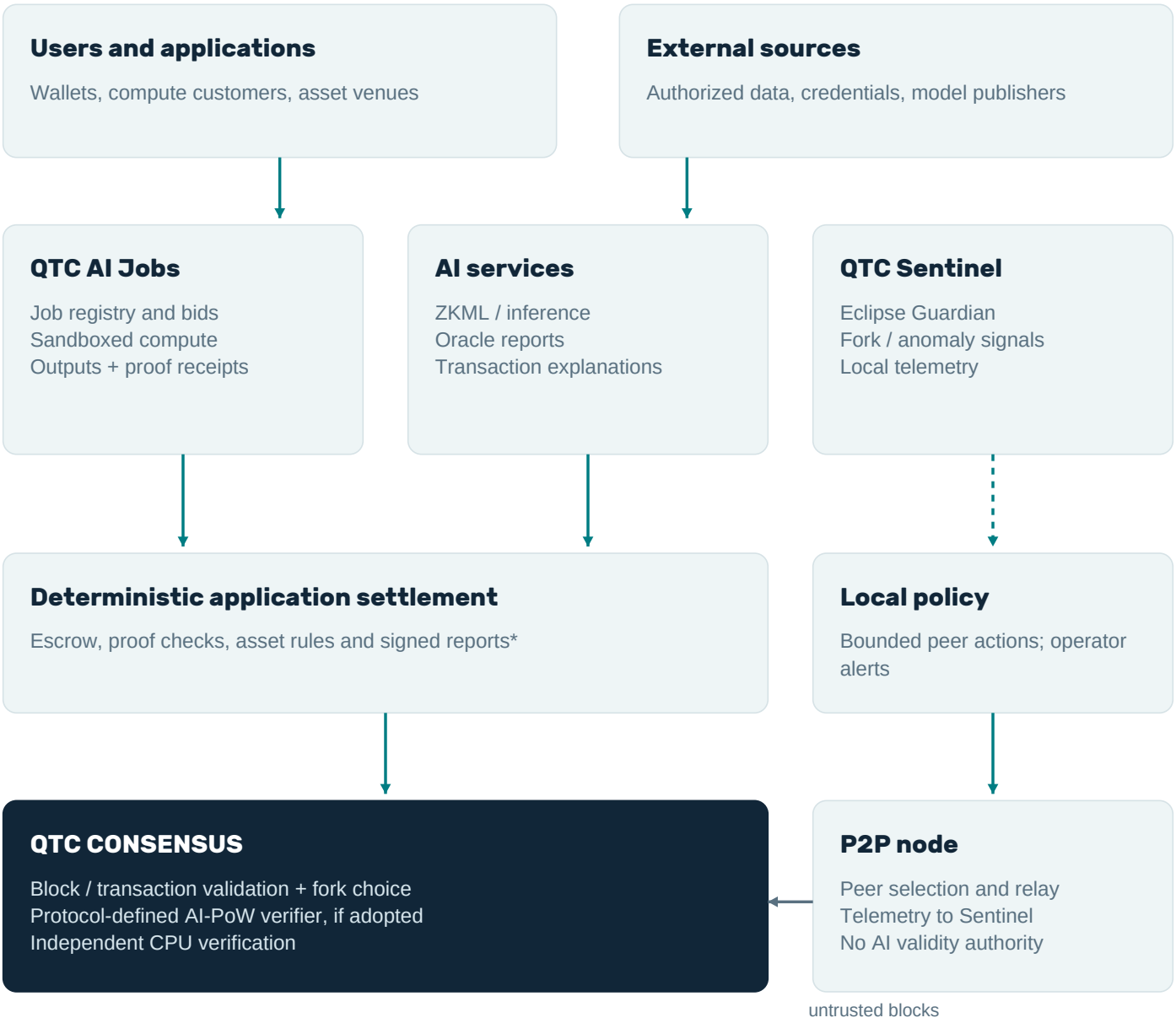
Architecture and compute: pages 4-6. Network and transaction security: pages 7-9. Hardware, proofs and oracles: pages 10-12. Assurance and delivery: pages 13-15.

The separation between valid work and fork choice follows the general proof-of-work principle that nodes validate blocks and select the qualifying chain under protocol rules [1]. QTC-specific rules still need to be confirmed.

03 / TARGET ARCHITECTURE

A deterministic core with isolated AI services

Solid arrows show data and settlement paths. Dashed arrows show advisory signals only.



* Application modules require supported scripts or a reviewed execution layer. They may validate compact receipts; they must never fetch a live model or remote job result during block validation. Customer workloads stay outside consensus execution.

04 / USEFUL AI COMPUTE AND AI-POW

Prove security before promising useful work

Matrix arithmetic is relevant to AI, but that alone does not make mining output commercially useful.

The target is expensive production, bounded verification and deterministic acceptance. A protocol-defined matrix computation could exploit GPUs or AI accelerators, but its resistance to shortcuts, precomputation and manipulated challenges must be established independently. A correct matrix product alone does not prove sufficient mining effort.

Consensus compute and commercial compute

Dimension	Consensus / AI-PoW	Commercial AI Jobs
Purpose	Sybil resistance and authenticated work.	Deliver a customer-defined result.
Work definition	Fixed by the activated protocol; bound to block context and nonce.	Chosen by a customer under an agreed job specification.
Availability	Every node can validate from bounded block data and protocol state.	Worker, model and data availability follow job terms.
Failure	Invalid work is rejected by deterministic rules.	Job fails, retries, disputes or refunds; chain validation continues.
Economics	Rewards and difficulty follow protocol rules.	Prices, deadlines and fees follow marketplace terms.

Requirements for any AI-PoW proposal

Bind and meter the work. Commit to the parent, header, nonce, arithmetic profile and witness. Define the target, authenticated work measure, anti-replay rules and all validation limits.

Analyze adversarial advantages. Test cheaper-than-claimed algorithms, solution reuse, challenge grinding, witness withholding, malicious dimensions and validation denial of service.

Measure CPU verification. Benchmark worst-case blocks, initial synchronization, bandwidth and memory. Exact replay may be deterministic yet expensive; cheap verification is a target, not an established QTC property.

> **NEAR-TERM CHOICE**
>
> Launch commercial AI Jobs beside the existing consensus system. Explore useful PoW separately. Do not make block production depend on customer demand, private inputs or a remote model service.

05 / INITIAL PRIORITY: QTC AI JOBS

Create a market for verifiable compute

Begin with narrow, reproducible workloads whose cost and correctness can be measured.

Start with exact matrix tasks and small, version-pinned inference models. Embeddings, scientific computation and model evaluation can follow once their acceptance rules are explicit. Distributed training is a later research track because data movement, coordination and verification can dominate cost.

A complete job lifecycle

Stage	Required behavior
1 / Specify and fund	Commit the model/program hash, input commitment, numerical profile, expected output schema, verification method, price, deadline and dispute terms. Fund a capped escrow.
2 / Match and execute	A provider accepts or bids. Run signed, allowlisted workloads in isolated workers with CPU/GPU, memory, time and network limits. Do not expose node or wallet keys.
3 / Commit and deliver	Bind the result and proof/receipt to the job ID, worker and destination. Make output retrievable under agreed retention terms. A hash alone does not establish delivery.
4 / Verify and settle	Use an approved proof verifier or disclosed replication/adjudication process. Pay exactly once after acceptance and any challenge window; expire, dispute or refund through defined states.

Make the assurance level visible

Cryptographic proof: verifies the encoded relation under the proof system's assumptions. **Independent recomputation:** provides a direct check for small tasks. **Replicated workers:** provide agreement under independence and quorum assumptions, not a mathematical guarantee. Label any operator-arbitrated pilot as such.

Protect economics, privacy and delivery

Price compute, proof generation, storage, bandwidth, verification and failed work separately. Compare net revenue after these costs and the opportunity cost of mining. Keep private inputs off-chain; encryption in transit or at rest does not hide plaintext from a worker executing it. Restrict early pilots to data with appropriate processing rights.

Pilot implementation: choose escrow only after reviewing QTC's scripts or VM. An explicitly trusted coordinator can support a prototype if native conditional settlement is unavailable; this must be disclosed to participants.

06 / INITIAL PRIORITY: QTC SENTINEL

Detect attacks without becoming an attack path

Deploy an optional local security companion with read-only access first.

Sentinel should correlate peer behavior, propagation, mempool pressure, malformed messages, fork observations and resource usage. Start with explainable statistical baselines and rules; add ML where it improves measured detection. An LLM can summarize incidents from structured evidence, but it should not control the node directly.

Detection and response pipeline

> **TELEMETRY → EVIDENCE → POLICY → BOUNDED ACTION**
>
> Telemetry is sanitized and rate-limited. Sentinel emits a signed or authenticated, versioned finding. Local deterministic policy checks objective conditions and connectivity constraints before any throttle, connection probe or disconnect. Operators review exceptional actions.

Observe	Produce	Permit only through policy
Peer latency, invalid-message counts, address churn, provider concentration.	A risk class, relevant time window, measured facts, confidence, model/policy version and expiry.	Rate limits for validated abuse; additional peer probes; time-limited actions with recorded reasons.
Block propagation, tip disagreement, validation backlog, memory pressure.	An incident record that separates observed evidence from inferred cause.	Alerts, bounded resource adjustments and operator escalation. Chain validity remains unchanged.

Controls that make Sentinel deployable

Least privilege. Use a read-only telemetry endpoint and a narrow action broker. Give the model no signing keys, shell access, configuration-write authority or unrestricted RPC credentials.

Resist manipulation. Treat peer strings, logs and job payloads as untrusted data. Use typed findings, independent feature checks, drift monitoring and replayable incident evidence. Never execute instructions found in telemetry.

Recover predictably. On timeout, model failure or missing data, return to baseline peer policy. Preserve minimum viable connectivity; do not bulk-disconnect because one score becomes high.

Release gate: shadow-mode replay and adversarial network tests must establish alert quality, resource overhead and recovery behavior before enabling actions. Publish false-positive and false-negative results for the evaluated scenarios.

07 / ECLIPSE GUARDIAN AND FORK DETECTION

Keep independent network views

Diversity is a defense signal; it is not proof that peers are honest or independent.

An eclipse attack isolates a node's view by dominating its connections [2]. Eclipse Guardian should monitor autonomous-system (ASN) and network-prefix concentration, address-source correlation, outbound churn and block-arrival patterns. Topology and route-inference errors complicate these defenses [3].

Eclipse Guardian: layered peer defense

Measure multiple forms of diversity. Track IPv4/IPv6 network groups, ASNs, operators, address provenance, connection age and supported transports. Geography is a weak supplemental signal; Tor or I2P identities are not automatic evidence of independence.

Diversify incrementally. Preserve working peers while probing candidates from varied discovery sources. Apply randomized selection, address-table protections, churn budgets and backoff. Keep deterministic peer-management safeguards effective when AI is disabled.

Avoid self-inflicted isolation. Never mass-ban an ASN solely because concentration is high. Confirm suspected abuse with objective events; keep operator recovery peers optional and avoid dependence on a single bootstrap service.

Fork and anomaly detection

Signal	Interpretation to investigate	Safe response
Tip disagreement or an unusual reorganization.	Normal propagation, a partition, incompatible software or an adversarial fork.	Collect headers and validated work; alert operators; preserve evidence.
Long inter-block interval or work-pattern change.	Chance variation, hash-rate change, connectivity failure or a protocol problem.	Compare independent observations. Do not infer fraud from one anomaly.
Propagation delay plus correlated peers.	Possible isolation or a shared network failure.	Probe diverse paths within policy limits; keep normal validation active.

CHAIN SELECTION REMAINS DETERMINISTIC

Sentinel never chooses the canonical fork. For a most-work design, compare only fully validated chains under the activated work rules. A venue may pause deposits or raise confirmation requirements under its own policy; that does not redefine the chain.

08 / TRANSACTION SECURITY AND ASSET APPLICATIONS

Make intent and authority visible before signing

Use deterministic simulation for effects, then AI to explain risk in plain language.

A wallet security layer should decode the exact transaction, simulate against an identified chain state and display asset changes, recipients, fees, approvals and contract calls. AI can translate the result and flag unusual patterns. Signing must bind to the exact transaction bytes and user-approved limits; a model explanation is never a signature authorization.

Transaction and smart-contract controls

Pre-signing review. Show spend amounts, allowance scope, recipients, slippage bounds, fees and expiry. Detect changed calldata or state and require renewed review where appropriate. Simulation cannot guarantee future execution because state and ordering may change.

Contract assurance. Combine AI-assisted review with tests, static analysis, invariant checks, fuzzing and independent audits. Inspect upgrade authority, access controls, reentrancy, arithmetic, oracle dependencies and emergency paths. AI review is not an audit substitute.

Relevance to tokenized securities

Function	AI contribution	Authoritative control
Participant eligibility	Triage onboarding exceptions and anomalies.	Authorized credentials, deterministic expiry/revocation checks and venue policy.
Market integrity	Detect suspicious wash-trading patterns, liquidity extraction and account compromise.	Investigations, enforceable venue controls and accountable human decisions.
Corporate actions and ownership	Reconcile event feeds and flag inconsistencies.	Issuer/agent records, signed instructions and defined legal rights.
Settlement and risk	Explain exposures; monitor delivery, collateral and oracle quality.	Asset/cash transfer rules, custody arrangements and reorganization policy.

A useful target is a public base chain with permissioned asset applications, subject to the selected legal and market structure. Token form alone does not establish shareholder rights or regulatory permission. SEC staff distinguishes issuer-sponsored and third-party tokenization models with different holder rights; its January 2026 statement is a staff view, not a rule [7].

Design boundary: AI cannot invent eligibility, waive restrictions, mint securities or replace the issuer, custodian or transfer agent. A launch requires a separate mapping of the chosen venue model to applicable requirements.

09 / HARDWARE PORTABILITY AND MINER OPTIMIZATION

Different hardware. The same validity result.

CPU verification is the reference; accelerators are optional implementations of the same specification.

Define canonical byte encoding, integer or finite-field arithmetic where appropriate, overflow behavior, rounding, reduction order and proof format. GPU-library reproducibility must not be assumed across vendors or versions: NVIDIA documents limits even within cuBLAS configurations [4]. QTC needs protocol-level arithmetic, not a library's default behavior.

Backend	Proposed role	Required evidence
CPU	Independent reference verifier and ordinary full-node path.	Known-answer vectors; worst-case latency, memory and synchronization benchmarks.
CUDA / NVIDIA	Mining, commercial jobs and optional accelerated verification.	Bit-identical acceptance/rejection versus the CPU reference across supported versions.
Metal / Apple	Alternative production and optional verification.	The same test corpus, serialization, overflow and edge-case behavior.
HIP / AMD	Alternative production and optional verification.	The same differential testing, malformed-proof handling and resource limits.

Verification design choices

Exact replay is straightforward to specify but may reproduce much of the original work. **Succinct proofs** can shift cost to the prover but add cryptographic and implementation assumptions. **Probabilistic matrix checks** require quantified soundness and grinding analysis; consensus cannot use unrelated random checks that let honest nodes disagree.

Optimize miners within fixed limits

ML can tune batching, scheduling, memory use, cooling, power settings, network placement and commercial job selection. Optimize accepted work per joule and net job revenue while respecting manufacturer limits, validation deadlines and operator policy. Reserve resources for node validation when miners share a host.

> **ACCEPTANCE CRITERION**
>
> A supported backend must agree with the reference on every conformance case. If it disagrees, disable that backend and diagnose the difference; never relax validity to accept its output. Report efficiency gains only after repeatable measurements, including stale work and proof overhead.

10 / ZKML AND VERIFIABLE INFERENCE

Prove execution; evaluate usefulness separately

A computation proof can bind a result to a specified model without making the model's judgment true.

Zero-knowledge machine learning (ZKML) can prove a relation between a model, an input and an output while hiding selected witness data. Research such as zkCNN demonstrates verifiable neural-network inference [5]. Applicability to QTC requires a defined circuit, security assumptions and measured proving and verification costs.

> **THE STATEMENT TO VERIFY**
>
> "For this job and protocol version, output Y was produced by model/program M on input X under numerical profile N." Commitments bind M and X; a public job identifier and destination prevent a proof being reused for another payment.

Design the full proof boundary

Pin the computation. Commit model weights, tokenizer, preprocessing, quantization, permitted operators and postprocessing. Proving a quantized model does not automatically prove equivalence to the original floating-point model.

Pin public inputs. Include the job ID, chain/domain identifier, model and input commitments, output commitment, verifier version and authorized recipient. Validate serialization and prevent replay.

Pin privacy and data access. Specify which fields are public and which remain private. Proofs can hide witness data from verifiers; they do not automatically hide inputs from the prover. Outputs and metadata may still leak information.

Pin the verifier lifecycle. Audit circuits and implementation; document trusted-setup requirements if any. Limit proof size and execution cost. Version verification keys and govern upgrades with explicit activation and recovery plans.

A sensible first application

Pilot one small inference model with public or approved non-sensitive inputs. Measure accuracy after quantization, proving time, proof size, CPU verification cost and cost per accepted result. Compare it with direct recomputation before selecting ZKML for production.

For eligibility, begin with signed credentials and deterministic validity, expiry and revocation checks. A later proof can establish possession of an acceptable credential without exposing all attributes. An AI risk score alone cannot establish identity, suitability or legal permission to trade.

What a proof does not establish: truth of source data, economic fairness, fitness for purpose, model accuracy in the real world or regulatory compliance. These require separate evidence and governance.

11 / QTC AI ORACLE NETWORK

Authenticate observations and constrain their use

Multiple models can share the same error. Source independence matters more than the model count.

The proposed oracle network should combine authorized data publishers, independent operators and deterministic application-level aggregation. AI can classify documents, compare sources and highlight anomalies. It must not invent a price, corporate action or eligibility decision that an application treats as authenticated fact.

A controlled reporting path

1 / Acquire and authenticate. Record source identity, observation time, instrument ID, currency, units and market state. Check signatures when available and validate data rights. Distinguish a vendor quote from an AI-derived estimate.

2 / Normalize and assess. Apply fixed schemas and deterministic normalization. AI may emit a separate risk annotation with model version, evidence and confidence; that annotation must not silently alter the source value.

3 / Aggregate and sign. Use independent operators, an explicit quorum and rules appropriate to the data type. Numeric feeds may use a robust median; corporate actions need authorized event provenance and exact identifiers.

4 / Verify and consume. Application code checks signatures, quorum, report version, sequence, freshness, bounds and replay protection. Define a clear unavailable state and the permitted fallback before going live.

Signed aggregate reports with on-chain signature checking are an established oracle pattern; Chainlink OCR is a useful reference [6]. This brief proposes a QTC design and does not imply a Chainlink integration or endorsement.

Security and economics

Failure mode	Required handling
Stale, conflicting or missing data	Pause the affected application action or use a narrowly specified fallback. Do not recycle an old value as fresh.
Collusion or shared upstream dependency	Assess operator ownership and upstream overlap; cap exposure and retain independent audit trails. A quorum can still be wrong.
Provable oracle misconduct	If bonding is used, define objective offenses, evidence, dispute timing and proportional penalties. Do not slash solely for model disagreement.

Consensus boundary: nodes verify the submitted report under deterministic application rules. They do not query websites or rerun an online model during block validation.

12 / SECURITY PRINCIPLES AND ASSURANCE

Contain failures at every layer

Security is a set of tested boundaries, not a property conferred by adding AI.

Threat or failure	Required control	Verification evidence
Model poisoning, evasion or prompt injection	Typed inputs, evidence checks, signed model releases, drift alarms and no direct model-to-command path.	Adversarial replay corpus; action broker rejects unsupported requests.
Malicious compute worker or customer job	Sandbox isolation, no node keys, resource limits, proof/recompute checks and explicit delivery terms.	Escape testing; invalid-output rejection; worker crashes leave validation available.
Forged, replayed or costly proofs	Domain separation, canonical encoding, job binding, verifier review and bounded verification costs.	Malformed-proof fuzzing; replay tests; worst-case validation measurements.
Peer capture or network partition	Layered discovery, measured diversity, churn limits, independent observations and recovery controls.	Simulated isolation, benign outages and recovery with AI disabled.
Key or administrator compromise	Separate signing, least privilege, hardware-backed keys where suitable, multiparty approval for critical changes.	Authorization review, key rotation and recovery drills.
Dependency or upgrade compromise	Pinned dependencies, reproducible releases where feasible, signed artifacts, review and staged rollout.	Build provenance; cross-platform vectors; downgrade/replay tests.

Rules for changes and emergencies

AI may draft proposals and simulate their impact. Humans review the specification, code, threat model and tests; nodes activate deterministic rules through the chosen governance process. Define voting or approval authority, activation conditions and compatibility before introducing any consensus change.

Emergency controls should stop or limit the affected marketplace, oracle or asset application through predeclared authority. They must not create an undocumented power to rewrite balances or declare an invalid block valid. A consensus repair is an explicit coordinated protocol change, not an AI action.

> **CRYPTOGRAPHIC CLAIMS REQUIRE THEIR OWN REVIEW**
>
> AI-native does not mean post-quantum secure. Inventory signatures, hashes, proof systems and key exchange separately; evaluate migration and compatibility before making any quantum-resistance claim.

13 / PRIORITIZED DEVELOPMENT ROADMAP

Deliver security and utility first

Illustrative sequence from project kickoff. Release gates, staffing and audits determine the actual schedule.

Stage	Work and ownership	Exit gate
Foundation Weeks 0-4	Protocol lead + security lead: verify the actual QTC chain, scripts/VM, arithmetic, APIs, keys and threat model. Establish testnet, telemetry and a CPU baseline.	Written compatibility map, authority boundaries and agreed performance/security budgets.
P0 / Sentinel Weeks 4-12	Security + node teams: ship read-only telemetry, incident evidence, Eclipse Guardian and fork alerts. Run in shadow mode.	Attack replay and benign-outage tests pass; measured alert quality and resource overhead; baseline works without AI.
P0 / AI Jobs Weeks 4-12	Compute + application teams: pilot one matrix task and one pinned inference model; implement worker isolation, verification and pay/refund states.	Valid jobs settle once; invalid/late jobs fail safely; delivery and dispute paths tested; unit economics measured.
P1 / Harden Months 3-6	Platform + security: complete backend conformance, enable bounded Sentinel actions, add wallet simulation and audit settlement.	No conformance mismatches; no unresolved critical/high findings in release scope; recovery drills pass.
P2 / Expand Months 6-12+	Cryptography + data + market teams: pilot small-model ZKML, signed oracles and one tokenized-asset use case.	Proof costs fit workload budgets; oracle failures contain risk; asset rights, controls and responsibilities documented.
Research gate No preset launch	Consensus researchers: evaluate useful AI-PoW, soundness, shortcut resistance, difficulty and hardware concentration.	Independent review, adversarial analysis and realistic CPU validation benchmarks before any activation proposal.

Metrics that decide whether to expand

Sentinel: detection delay, false positives/negatives, connectivity retained and CPU/RAM overhead. **AI Jobs:** completion rate, verification cost, dispute/refund rate and net provider margin. **Protocol:** worst-case verification latency, synchronization cost, reorganization behavior and backend agreement.

IMMEDIATE EXECUTIVE DECISION

Assign accountable owners and fund the two P0 tracks. Keep consensus changes outside the initial product scope. Require an evidence review before enabling autonomous security actions or accepting production marketplace funds.

14 / EVIDENCE AND NEXT DECISIONS

Reference notes and implementation gates

Sources support technical context. The QTC architecture, controls and roadmap are design recommendations.

Source basis: the immediately preceding AI architecture answer in “Developing QTS For Tokenized Stocks,” supplied through the referenced conversation and retrieved in full. This brief develops that answer into an implementation-oriented proposal; it makes no claim that the named QTC subsystems are deployed.

[1] Satoshi Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System (2008).
Proof-of-work validation and chain-selection background.

[2] Heilman et al., Eclipse Attacks on Bitcoin’s Peer-to-Peer Network, USENIX Security (2015).
Connection capture and eclipse attack mechanisms.

[3] Tran, Shenoi and Kang, On the Routing-Aware Peering against Network-Eclipse Attacks in Bitcoin, USENIX Security (2021).
Limits of simple routing/diversity defenses.

[4] NVIDIA, cuBLAS documentation, section 2.1.4: Results Reproducibility.
Version, architecture and execution-mode constraints.

[5] Liu, Xie and Zhang, zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy (CCS 2021; revised 2023).
A primary research example of verifiable neural-network inference.

[6] Chainlink documentation, Offchain Reporting.
Off-chain aggregation and on-chain quorum-signature verification.

[7] SEC staff, Statement on Tokenized Securities (28 January 2026).
Taxonomy and holder-rights distinctions; staff statement, not a rule.

Public references accessed 24 September 2026. Titles above are clickable.

Resolve before implementation

Confirm QTC/QTS naming and chain identity; obtain the current consensus and execution specifications; select the first job and verification method; define settlement trust and dispute authority; approve node resource budgets; and agree who can release models, change policies, sign oracle reports and operate emergency controls.

> **SUCCESS CONDITION**
>
> QTC gains useful compute services and stronger operational defenses while every full node retains the ability to decide validity from explicit, reproducible protocol rules.